

PARADIGMA ORIENTADO A OBJETOS

UNIDAD N° I

Objetos en el Mundo



SEMANA 1

Consideraciones previas

El contenido que se expone a continuación está ligado a los siguientes objetivos:

- Distingue los distintos tipos de aplicaciones informáticas de acuerdo con su enfoque y características.
- Distingue las distintas formas de programación y sus diferentes características para la aplicación de estos paradigmas.
- Distingue los conceptos de objetos, características y comportamiento para entender la base del paradigma.
- Distingue el concepto de paradigma orientado a objetos.
- Distingue el software a utilizar y el método Main() o método principal como método de ejecución del código para su compilación.
- Aplica los conceptos básicos de la orientación a objetos utilizando aplicación de desarrollo de software.

Sobre las fuentes utilizadas en el material

El presente Material de Estudio constituye un ejercicio de recopilación de distintas fuentes, cuyas referencias bibliográficas estarán debidamente señaladas al final del documento. Este material, en ningún caso pretende asumir como propia la autoría de las ideas planteadas. La información que se incorpora tiene como única finalidad el apoyo para el desarrollo de los contenidos de la unidad correspondiente, respetando los derechos de autor ligados a las ideas e información seleccionada para los fines específicos de cada asignatura.

Introducción

En esta primera semana se darán los primeros pasos en este interesante mundo de la programación. Para ello, se analizarán los distintos tipos de aplicaciones informáticas, junto a los diversos tipos de programación que el mercado ofrece hoy en día. Posterior a ello se hará énfasis en el paradigma orientado a objetos. Por último, se revisará la instalación del programa Greenfoot, y cómo crear nuevos proyectos.

Los conceptos de esta semana son de vital importancia, ya que permitirán tener un acercamiento inicial al desarrollo de aplicaciones, usando para ello una plataforma didáctica, basando el aprendizaje en el concepto de la creación de videojuegos. Es necesario recalcar que se comenzará con casos sencillos, sin dejar de mencionar las bases teóricas del lenguaje de programación.

Finalmente, se recomienda poner atención al proceso de instalación de los programas y complementos que se indiquen en las actividades, ya que es fundamental para el desarrollo de los ejercicios y evaluaciones posteriores.

Ideas fuerza

1. **Aplicación:** es un programa informático diseñado como una herramienta para realizar operaciones específicas. Son diseñadas para facilitar tareas complejas y hacer más sencilla la experiencia informática de las personas.
2. **Programación:** es la actividad asociada al desarrollo de aplicaciones o programas informáticos, ya sea con fines educativos, personales o empresariales. En algunos casos se representa como el proceso de crear código fuente en un determinado lenguaje.
3. **Paradigma:** es una manera o estilo de programación de software. Existen diferentes lenguajes de programación y varios modos de trabajar para obtener los resultados que necesitan los programadores. Se trata de un conjunto de métodos sistemáticos aplicables en todos los niveles del diseño de programas para resolver problemas computacionales.
4. **Java:** es lenguaje de programación que se basa en el paradigma orientado a objetos. Es usado para crear aplicaciones web, móviles o de escritorio. Su descarga y uso son gratuitas para fines personales, y actualmente su licencia y derechos pertenecen a la empresa Oracle.
5. **JRE:** Java Runtime Environment, o JRE, es una capa de software que se ejecuta encima del software del sistema operativo de una computadora y proporciona las bibliotecas de clases y otros recursos que un programa Java específico necesita ejecutar.
6. **JVM:** es un entorno de tiempo de ejecución que se puede añadir en un navegador web o en cualquier sistema operativo. Ejecuta instrucciones generadas por un compilador Java. Consta de un intérprete de bytecode y un entorno de tiempo de ejecución que permiten ejecutar los archivos de clase Java en cualquier plataforma, sea cual sea la plataforma en la que se desarrollaron originariamente.
7. **Greenfoot:** es un entorno de desarrollo integrado que permite crear aplicaciones basadas en lenguaje Java, usando el concepto de los juegos. Su orientación es más bien pedagógica, lo cual no implica que no se puedan generar aplicaciones ejecutables

Índice

Contenido

<i>1.- Tipos de aplicaciones informáticas</i>	<i>6</i>
1.1.- Web	6
1.2.- Escritorio	6
1.3.- Mobile.....	7
<i>2.- Tipos de programación.....</i>	<i>8</i>
2.1.- Funcional.....	8
2.2.- Orientada a eventos.....	9
2.3.- Orientada a objetos	9
<i>3.- ¿Qué es un paradigma?.....</i>	<i>9</i>
<i>4.- Paradigma orientado a objetos</i>	<i>10</i>
4.1.- ¿Qué es?.....	10
4.2.- Abstracción y programación	11
<i>5.- Software a usar</i>	<i>13</i>
5.1.- Conociendo su interfaz.....	13
5.2.- Método Main() o principal	15
5.3.- Creando una clase de ejemplo.....	15

Desarrollo

1.- Tipos de aplicaciones informáticas

Un programa o aplicación informática [1] es la definición precisa de una tarea de computación, siendo el propósito de un programa su ejecución en un procesador, y suponiendo dicha ejecución cierto cómputo o transformación.

Para poder escribir programas de forma precisa y no ambigua es necesario definir reglas que determinen tanto lo que se puede escribir en un programa (y el procesador podrá interpretar) como el resultado de la ejecución de dicho programa por el procesador. Dicha notación, conjunto de reglas y definiciones es lo que se denomina un lenguaje de programación.

Dentro de las opciones de lenguajes de programación, existen algunos que se orientan a tipos de dispositivos específicos. Asimismo, hay otros que se adaptan a diversos escenarios. Es así como las aplicaciones que se pueden desarrollar se pueden dividir en categorías: web, escritorio y mobile.

1.1.- Web

Las aplicaciones web son programas que funcionan en internet. Es decir, que los datos o los archivos en los que se trabaja son procesados y almacenados dentro de un servidor. Estas aplicaciones, por lo general, no necesitan ser instaladas en un computador para ser ocupadas.

El concepto de aplicaciones web está relacionado con el almacenamiento en la nube. Toda la información se guarda de forma permanente en grandes servidores de internet y envían, a un dispositivo o equipo, los datos que requieres en ese momento, dejando una copia temporal dentro del equipo visitante.

1.2.- Escritorio

Las aplicaciones de escritorio son el conjunto de programas o herramientas que están instalados en un computador de escritorio o portátiles, y que únicamente es posible usarlos en dichos dispositivos. No es posible trasladarlos a otro equipo o acceder a ellos de manera sencilla desde otro equipo, por lo que posee esta limitación.

A la hora de desarrollar una aplicación de escritorio, un desarrollador se enfrenta al dilema de qué lenguaje utilizar para que la aplicación se convierta en lo más multiplataforma posible. Por lo mismo, una de las maneras de tratar la portabilidad es a través de estos lenguajes.

1.3.- Mobile

Una aplicación móvil o “mobile”, también llamada app móvil, es un tipo de aplicación diseñada para ejecutarse en un dispositivo móvil, que puede ser un teléfono inteligente o una tableta. Incluso si las aplicaciones suelen ser pequeñas unidades de software con funciones limitadas, se las arreglan para proporcionar a los usuarios servicios y experiencias de calidad.

A diferencia de las aplicaciones diseñadas para computadoras de escritorio, las aplicaciones móviles se alejan de los sistemas de software integrados. En cambio, cada aplicación móvil proporciona una funcionalidad aislada y limitada. Por ejemplo, puede ser un juego, una calculadora o un navegador web móvil.

ANTES DE CONTINUAR CON LA LECTURA...REFLEXIONEMOS

Si se necesita crear una aplicación para un negocio local para repartos a domicilio, la que debe ser accesible rápidamente ¿qué tipo de aplicación podría satisfacer esta necesidad?



2.- Tipos de programación

El propósito principal de la programación consiste en describir la solución computacional (eficiente) de clases de problemas. Aunque hay que destacar que se ha demostrado la existencia (le problemas para los que no puede existir solución computacional alguna, lo que implica una limitación importante a las posibilidades de la mecanización del cálculo.

2.1.- Funcional

Se basa en el concepto matemático de función y se expresa de manera declarativa. Las funciones podrán ser asignadas a variables además podrán ser utilizadas como entrada y salida de otras funciones. Ejemplos de lenguajes de programación que se basan en este concepto son Lisp, SML, Hope, Hasckel.

2.2.- Orientada a eventos

La programación orientada a eventos es un paradigma de programación en el que la estructura y la ejecución de los programas van determinados por los sucesos o acciones que ocurren en el sistema, definidos por el usuario o por el propio sistema.

La programación orientada a eventos es muy fácil de usar y es adecuada para aquellas personas que tienen poco conocimiento en programación. Con los lenguajes orientados a eventos se pueden realizar en poco tiempo aplicaciones sencillas y muy funcionales, utilizando interfaces gráficas en las que se insertan componentes o controles a los que se le programan eventos. Dichos eventos permiten al usuario realizar una serie de acciones lógicas para un determinado programa.

2.3.- Orientada a objetos

En este modelo de paradigma se construyen modelos que representan elementos (objetos) del problema a resolver, que tienen características y funciones. Permite separar los diferentes componentes de un programa, simplificando así su creación, depuración y posteriores mejoras. La programación orientada a objetos disminuye los errores y promueve la reutilización del código. Es una manera especial de programar, que se acerca de alguna manera a cómo se expresan las cosas en la vida real.

En la programación orientada a objetos, las acciones se expresan en forma de relaciones entre objetos y responsabilidades de cada uno. Es una manera de acercar los conceptos del mundo real al desarrollo de aplicaciones. Ejemplos de lenguajes de programación que utilizan esta idea son SmallTalk y Java.

3.- ¿Qué es un paradigma?

Un paradigma de programación [2] es un modelo conceptual para desarrollar programas. El uso de un paradigma se refuerza por el lenguaje que se escoja para realizar un programa concreto, aunque en general, con mayor o menor dificultad, se puede usar cualquier lenguaje de programación para seguir cualquier paradigma.

Dicho de otra manera, un paradigma de programación es una manera o estilo de programación de software. Existen diferentes formas de diseñar un lenguaje de programación y varios modos de trabajar para obtener los resultados que necesitan los programadores. Se trata de un conjunto de métodos sistemáticos aplicables en todos los niveles del diseño de programas para resolver problemas computacionales.

Los lenguajes de programación adoptan uno o varios paradigmas en función del tipo de órdenes que permiten implementar como, por ejemplo, Python o JavaScript, que son multiparadigmas.

4.- Paradigma orientado a objetos

Antes de hablar de programación orientada a objetos es necesario definir que es un objeto. Un objeto es algo a lo que se le puede enviar mensajes y que puede responder a los mismos y que tiene un estado, un comportamiento bien definido y una identidad. El estado de un objeto está definido por el valor de ciertas variables internas al objeto. Este estado puede cambiar dependiendo de los mensajes que reciba desde el exterior o de un cambio interno al propio objeto. El comportamiento de un objeto varía en función del estado en el que se encuentra, y se percibe por los valores que devuelve ante los mensajes que recibe y por los cambios que produce en los objetos con los que se relaciona. La identidad de un objeto es aquello que lo hace distinguible de otros.

4.1.- ¿Qué es?

El paradigma o programación orientado a objetos es un método de desarrollo en el cual los programas se organizan como colecciones de objetos que cooperan para resolver un problema. En general los objetos pueden corresponderse a entidades del mundo real (como un coche o un gato), a acciones (como saltar o realizar una transacción bancaria) o a procesos (como el vuelo o el aprendizaje).

La programación orientada a objetos se basa en el modelo de objetos. Este modelo se fundamenta en el uso de 7 capacidades, 4 de las cuales se consideran principales y 3 secundarias. Los lenguajes de programación orientados a objetos se caracterizan porque proporcionan mecanismos que dan soporte a estas capacidades.

Antes de continuar es necesario comprender los conceptos asociados con este paradigma:

- **Clase:** Es una serie de atributos y métodos que definen comportamientos en los objetos. Es decir, es un diseño, un molde, que puede producir uno o muchos objetos.
- **Atributo:** Es una característica que es común a todos los objetos de una clase, por lo cual se declara un elemento de la clase. Cada atributo debe tener un tipo dependiendo de su naturaleza y objetivo.
- **Método:** Es un comportamiento que tiene un objeto. Una clase puede tener muchos métodos. Se pueden activar en el momento y orden deseado para controlar el comportamiento del objeto, y además a través de ellos se puede modificar los valores de los atributos.
- **Objeto:** Un objeto es una entidad que tiene sus propias características y comportamientos. Un objeto en Java es el producto de una clase, partiendo de la base que esta última es solo un diseño o molde.

Imaginemos que se tiene la clase Vehículo. Sus características (atributos) podrían ser, por ejemplo, que tiene ruedas y pedales. Sus comportamientos (métodos) pueden ser acelerar o frenar. A partir de esta descripción, es posible crear varios vehículos diferentes que compartan estas características. Por ejemplo, un auto, una camioneta o un furgón.

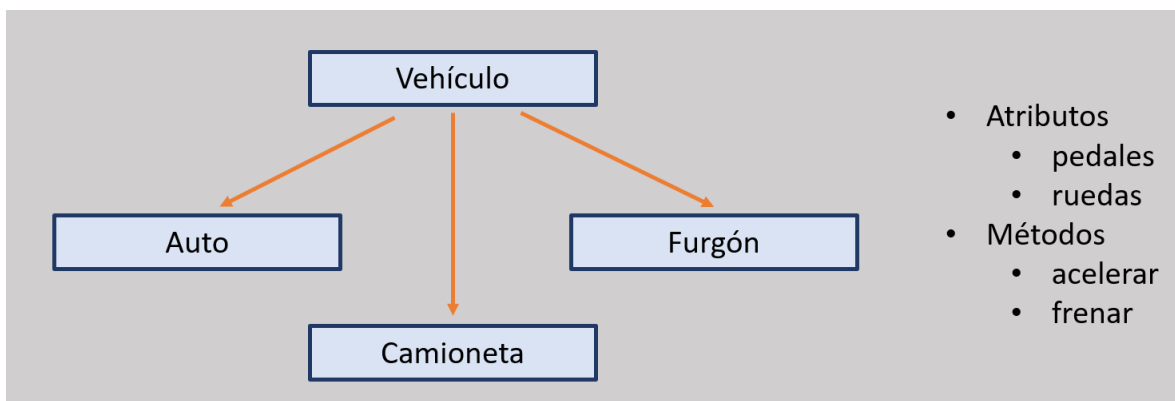


Ilustración 1: Representación de un vehículo como una clase

Las capacidades principales son: abstraer, encapsular, modularizar y jerarquizar. Las capacidades secundarias, en tanto, son tipo, concurrencia y persistencia.

4.2.- Abstracción y programación

Abstraer [2] es la capacidad que permite distinguir aquellas características fundamentales de un objeto que lo hacen diferente del resto, y que proporcionan límites conceptuales bien definidos relativos a la perspectiva del que lo visualiza. La abstracción surge de reconocer las similitudes entre objetos, situaciones o procesos en el mundo real, y la decisión de concentrarse en esas similitudes e ignorar las diferencias. Así, una abstracción se focaliza sobre una posible vista, ayudando a separar el comportamiento esencial de un objeto de su implementación

Imagina el caso de los mapas; un mapa político representa la división en regiones de la superficie de un país; si el mapa es físico, su foco estará en los aspectos geográficos de la misma superficie. En el caso de la programación, los lenguajes de orientados a objetos facilitan la abstracción gracias a que permiten definir interfaces comunes para comunicarse con clases de objetos. Estas interfaces están compuestas por los métodos, que son funciones que pueden aplicarse sobre el objeto y que pueden verse como los mensajes que es posible enviar al objeto. Normalmente un objeto puede tener varias interfaces.

ANTES DE CONTINUAR CON LA LECTURA...REFLEXIONEMOS

En el contexto del aprendizaje de la orientación a objetos, ¿por qué será recomendable analizar este concepto como un paradigma antes de aprender los conceptos específicos de un lenguaje?

Si se necesita crear una clase que represente a todas las personas de un mismo país ¿qué atributos se deberían considerar?



5.- Software a usar

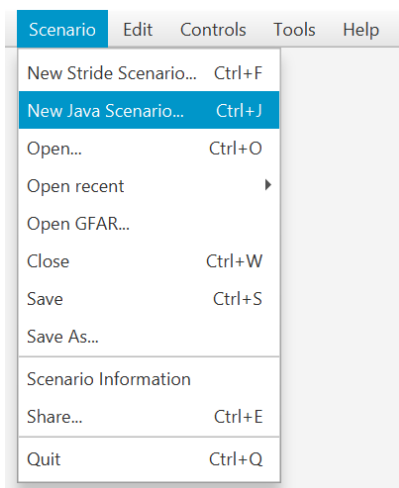
En esta asignatura la base del aprendizaje será el paradigma orientado a objetos. Es por ello por lo que se usará una herramienta que permita el entendimiento de los conceptos de manera lúdica, más que el desarrollo de aplicaciones de uso masivo. La aplicación que se utilizará se llama Greenfoot.

Greenfoot [3] es una herramienta de software diseñada para permitir que las personas que dan sus primeros pasos en informática puedan obtener experiencia en la programación orientada a objetos. Esta plataforma soporta el desarrollo de aplicaciones gráficas similares a juegos, con actores, mundos y opciones de interacción entre elementos.

Greenfoot fue diseñado e implementado en la Universidad de Kent, Inglaterra y la Universidad Deakin, Melbourne, Australia. La base para el desarrollo de escenarios es el lenguaje Java. Por lo mismo, antes de instalar Greenfoot en un equipo local, es necesario instalar el Kit de Desarrollo de Java, o JDK; siempre es recomendable instalar la última versión existente en la página oficial del fabricante [4].

5.1.- Conociendo su interfaz

Después de instalar el Kit de Desarrollo de Java, puedes instalar Greenfoot [5]. Su uso es liberado, y se puede usar en los sistemas operativos más comunes. Ya instalado el programa, se debe iniciar normalmente.



Si es la primera vez que se abre Greenfoot, aparecerá un cuadro de diálogo que consulta al usuario qué quiere hacer. Se debe seleccionar la opción “Elegir un escenario”; en otro caso, se puede usar en el menú principal la opción Escenario → Abrir. Otra opción es crear un nuevo escenario Java; para hacer esto se debe acceder al menú principal en la opción Escenario → Nuevo escenario Java.

Al crear un nuevo proyecto o abriendo uno existente, el usuario verá la interfaz de la ventana principal de Greenfoot, con el escenario abierto, de

forma similar a lo que se muestra en la imagen siguiente. La ventana principal consta de tres áreas principales y un par de botones adicionales. Las áreas principales son:

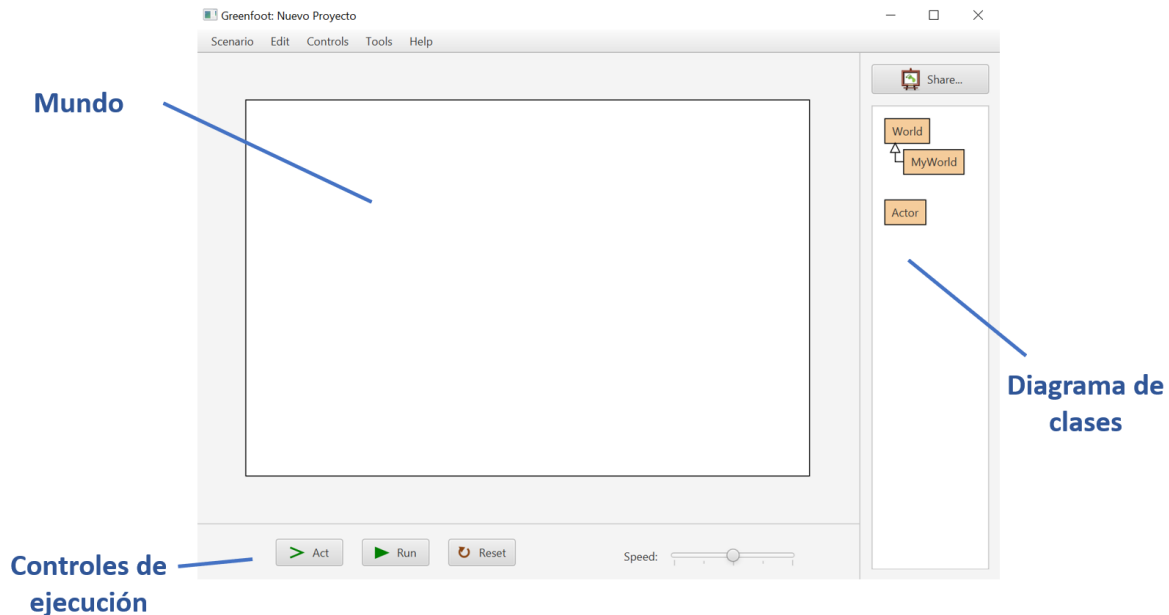
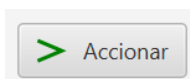


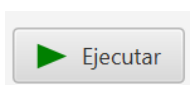
Ilustración 2: La ventana principal de Greenfoot

- **Mundo:** Se llama mundo al área más grande que ocupa la mayor parte de la pantalla (un fondo blanco en este caso). Es donde el programa se ejecutará y el usuario verá lo que ocurre.
- **Diagrama de clases:** Se llama diagrama de clases al área de la derecha con cajas y flechas. En ellas se puede crear a los personajes y sus comportamientos, así como cambiar el tamaño, apariencia y características del fondo.
- **Controles de ejecución:** Los controles de ejecución son los botones Accionar, Ejecutar, y Reiniciar y la barra inferior para ajustar la velocidad.

Dentro de los controles de ejecución, cada botón tiene funciones específicas en el desarrollo. A continuación, se indica la función de cada uno.

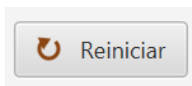


Botón Act: Este botón permitirá ver un paso a paso de lo que está ocurriendo en el juego, y comprobar cómo está funcionando.

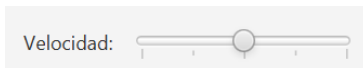


Botón Run: Hace que corra el juego, es decir, lo inicia para que sea posible jugarlo y ver su

funcionamiento. A diferencia del botón Act, no se verá el paso a paso.



Botón Reset: Este botón hará que el juego regrese a su estado inicial.



Control Speed: Con este control deslizante se podrá aumentar o disminuir la velocidad a la que corre el juego.

5.2.- Método Main() o principal

En el lenguaje Java, una clase representa un objeto del mundo real o un concepto. Esta entidad tendrá atributos y métodos, y en un proyecto cada clase será un archivo independiente con la extensión .java.

Toda clase por si sola no ejecutará acciones mientras no sea instanciada en un objeto. Por lo mismo, para poder crear objetos y llamar a sus métodos, se debe crear una clase que tenga un método main. Por ejemplo, si se desea crear un programa en Java que retorne un mensaje por consola, se vería de la siguiente manera:

```
public class PrimerPrograma {  
    public static void main(String[] args) {  
        System.out.println("Hola a todos");  
    }  
}
```

En el caso de Greenfoot, no es necesario agregar dicho método en alguna clase, ya que cuenta con métodos personalizados para realizar acciones. En este caso el método se llama act(); a partir del mismo se puede llamar a todas las demás acciones indicadas en la misma clase o en otras presentes en el proyecto.

5.3.- Creando una clase de ejemplo

A continuación, se indicará un ejemplo paso a paso que permitirá crear un nuevo proyecto y una clase dentro de éste. Los pasos son los siguientes:

- A. Abrir el programa Greenfoot.
- B. En el menú principal, seleccionar la opción “Escenario → Nuevo escenario Java”.
- C. Se desplegará una ventana en la que se solicitará un nombre de proyecto. A la vez, se debe seleccionar una ruta dentro del equipo local en la que guardar el nuevo proyecto. Al presionar el botón guardar se creará el nuevo proyecto.
- D. En la sección de la derecha aparecerá una clase de nombre MyWorld; para definir una nueva imagen de fondo se debe seleccionar esta clase con el botón secundario del ratón, y presionar la opción “Elegir imagen ...”.
- E. Aparecerá una nueva ventana con un listado de imágenes. Desde la sección “backgrounds” se debe seleccionar una de las imágenes y presionar el botón “Aceptar”.
- F. A un costado de la clase “myWorld” aparecerá una imagen de referencia. En este punto el fondo seguirá blanco, por tanto, si se desea aplica un fondo, se debe presionar con el botón secundario del mouse la clase MyWorld, y seleccionar la opción “new MyWorld()”.
- G. Para crear una clase o actor en el mundo, es necesario posicionar el mouse en la clase Actor, y con el botón secundario del mouse elegir la opción “New subclass ...”.
- H. En la ventana que aparecerá, se debe indicar un nombre de la clase, manteniendo la opción “Java” que aparecerá a un costado. Al mismo tiempo se debe seleccionar una imagen en alguna de las categorías. En este ejemplo se seleccionará desde la categoría “transport” la imagen de un cohete. Una vez realizado lo anterior se debe presionar el botón “Aceptar”.
- I. La nueva subclase creada aparecerá bajo la clase Actor. Si se desea aplicar acciones sobre ésta se debe hacer doble clic. En la ventana que se abrirá es posible agregar nuevos métodos, y modificar lo existente en el método act().
- J. Para agregar un nuevo elemento u objeto al mundo, se debe hacer clic con el botón secundario sobre la clase anteriormente creada, y presionar la opción “new Cohete()”. Aparecerá una imagen, la cual debe ser posicionada en algún lugar del mundo.
- K. Si se desea guardar las instancias de clases creadas en el mundo, se debe seleccionar desde el menú principal la opción “Herramientas → Guardar el mundo”.

La imagen que se muestra a continuación representa cómo se debería ver el proyecto obtenido.

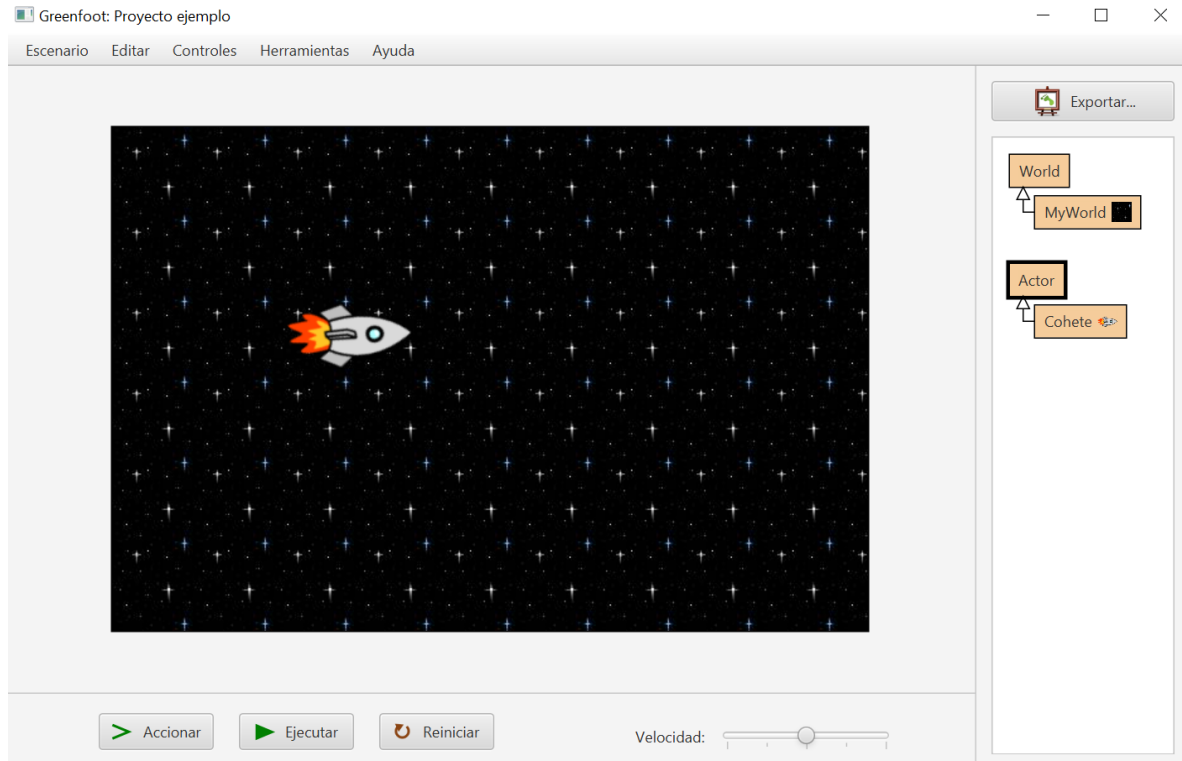


Ilustración 3: Proyecto de ejemplo

ANTES DE CONTINUAR CON LA LECTURA...REFLEXIONEMOS

Además de Greenfoot ¿existirán otros programas o entornos de desarrollo integrado (IDE) que permitan desarrollar?

Si se desea desarrollar aplicaciones en lenguaje Java ¿es obligatorio contar con un programa que permita la codificación o ejecución?



Conclusión

En esta primera semana se han dado los primeros pasos en el mundo de la programación orientada a objetos, realizando un estudio desde su visión como paradigma. Por lo mismo, se ha introducido el programa Greenfoot, que permite internalizar estos conceptos desde la práctica, por medio de una metodología didáctica.

Antes de eso, se ha realizado un análisis de conceptos ligados al mundo de la programación, como son los distintos tipos de aplicaciones y diferentes paradigmas de programación.

Asimismo, se han indicado los pasos esenciales que permiten crear un proyecto en Greenfoot, agregar un fondo y crear nuevos actores al mundo.

En la siguiente semana se analizarán más conceptos de este paradigma, analizándolos siempre desde la perspectiva que nos ofrece el entorno de desarrollo. Antes de terminar, se plantean las siguientes interrogantes:

- ¿Qué otras acciones debiese permitir Greenfoot desde la perspectiva de la creación de un juego?
- Si es necesario que un actor se mueva ¿en qué parte del código debería indicarse la instrucción?

Bibliografía

- [1] Prieto Saez, N. y Casanova Faus, A. (2016). Empezar a programar usando Java (3a. ed.). Valencia, España: Editorial de la Universidad Politécnica de Valencia.
- [2] Vélez Serrano, J. (2011). Diseñar y programar, todo es empezar: una introducción a la Programación Orientada a Objetos usando UML y Java. Madrid, Spain: Dykinson.
- [3] Kolling, M. (30 de enero de 2022). “Greenfoot tutorial”, <https://www.greenfoot.org/files/translations/Spanish/tutorial.html>
- [4] Descargas de Java | Oracle. (30 de enero de 2023), <https://www.oracle.com/java/technologies/downloads/>
- [5] Descarga de Greenfoot (30 de enero de 2023), <https://www.greenfoot.org/download>

